

Module 8.1 - Component Communication Overview

Learning Objectives: Understand the different methods for component interaction in Angular, implement parent-to-child communication using @Input decorator, implement child-to-parent communication using @Output and EventEmitter, and use ViewChild decorator for direct access to child components and DOM elements.

1. Understanding Component Communication

1.1 Why Component Communication?

In Angular applications, components are organized in a tree-like hierarchy. Components often need to share data and coordinate their behavior. Component communication provides structured ways to pass data and events between components.

Common Communication Scenarios:

- **Parent → Child:** Pass configuration, data, or settings from parent to child
- **Child → Parent:** Notify parent about events, user actions, or state changes
- **Sibling → Sibling:** Share data between components at the same level
- **Unrelated Components:** Communication between components in different parts of the app

Component Tree Example:

```
AppComponent (root)
├── HeaderComponent
│   ├── LogoComponent
│   └── NavigationComponent
└── MainComponent
    ├── SidebarComponent
    └── ContentComponent
```



1.2 Communication Methods Overview

Method	Direction	Use Case	Decorator/API
@Input	Parent → Child	Pass data down the component tree	@Input()
@Output	Child → Parent	Emit events up the component tree	@Output() + EventEmitter
ViewChild	Parent → Child	Direct access to child component/element	@ViewChild()
ViewChildren	Parent → Children	Access multiple child components	@ViewChildren()
Service	Any → Any	Share data between unrelated components	Shared Service + DI

2. Parent to Child Communication with @Input

2.1 Understanding @Input Decorator

The `@Input` decorator marks a class property as an input property, allowing parent components to bind data to it. This is the primary way to pass data from parent to child components.

Key Concepts:

- **Property Binding:** Parent binds to child's input property using `[propertyName]="value"`
- **One-Way Data Flow:** Data flows from parent to child (unidirectional)
- **Change Detection:** Child component updates when parent data changes

- **Type Safety:** Input properties are type-checked by TypeScript

2.2 Basic @Input Example

Child Component (user-card.component.ts):

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-user-card',
  template: `
    <div class="user-card">
      <h3>{{ userName }}</h3>
      <p>Age: {{ userAge }}</p>
      <p>Role: {{ userRole }}</p>
    </div>
  `,
  styles: [
    `.user-card {
      border: 1px solid #ddd;
      padding: 16px;
      border-radius: 8px;
      margin: 10px 0;
    }`
  ]
})
export class UserCardComponent {
  // Input properties - can receive data from parent
  @Input() userName: string = '';
  @Input() userAge: number = 0;
  @Input() userRole: string = 'User';
}
```

Parent Component (app.component.ts):

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `
    <h2>User Directory</h2>

    <!-- Passing data to child component via @Input properties -->
  `
})
export class AppComponent {
}
```

```
<app-user-card
  [userName]='John Doe'
  [userAge]="28"
  [userRole]='Developer'>
</app-user-card>

<app-user-card
  [userName]='Jane Smith'
  [userAge]="32"
  [userRole]='Designer'>
</app-user-card>

<!-- Using component properties -->
<app-user-card
  [userName]="adminName"
  [userAge]="adminAge"
  [userRole]="adminRole">
</app-user-card>
,
})
export class AppComponent {
  adminName = 'Alice Johnson';
  adminAge = 35;
  adminRole = 'Administrator';
}
```

User Directory

John Doe

Age: 28

Role: Developer

Jane Smith

Age: 32

Role: Designer

Alice Johnson
Age: 35
Role: Administrator

2.3 @Input with Complex Data Types

Input properties can accept any data type: primitives, objects, arrays, or even functions.

Interface Definition:

```
interface Product {  
  id: number;  
  name: string;  
  price: number;  
  category: string;  
  imageUrl: string;  
  inStock: boolean;  
}
```

Child Component (product-card.component.ts):

```
import { Component, Input } from '@angular/core';  
  
@Component({  
  selector: 'app-product-card',  
  template: `  
    <div class="product-card" *ngIf="product">  
      <img [src]="product.imageUrl" [alt]="product.name">  
      <h3>{{ product.name }}</h3>  
      <p class="price">{{ product.price | currency:'USD' }}</p>  
      <p class="category">{{ product.category | uppercase }}</p>  
      <p class="stock" [class.out-of-stock]="!product.inStock">  
        {{ product.inStock ? 'In Stock' : 'Out of Stock' }}  
      </p>  
    </div>  
  `,  
  styles: [`  
    .product-card {  
      border: 1px solid #e0e0e0;  
      padding: 16px;  
    }  
  ]  
})
```

```
border-radius: 8px;
max-width: 300px;
}
.price { font-size: 1.5em; color: #2c3e50; font-weight: bold; }
.category { color: #7f8c8d; font-size: 0.9em; }
.stock { color: green; font-weight: 600; }
.out-of-stock { color: red; }
`]
})
export class ProductCardComponent {
  @Input() product!: Product; // Definite assignment with !
}
```

Parent Component:

```
export class ProductListComponent {
  products: Product[] = [
    {
      id: 1,
      name: 'Wireless Headphones',
      price: 149.99,
      category: 'Electronics',
      imageUrl: 'assets/headphones.jpg',
      inStock: true
    },
    {
      id: 2,
      name: 'Office Chair',
      price: 299.99,
      category: 'Furniture',
      imageUrl: 'assets/chair.jpg',
      inStock: false
    }
  ];
}
```

Parent Template:

```
<h2>Product Catalog</h2>

<div class="product-grid">
  <app-product-card
    *ngFor="let item of products">
```

```
[product]="item">
  </app-product-card>
</div>
```

Product Catalog

[Image: Headphones]
Wireless Headphones
\$149.99
 ELECTRONICS
In Stock

[Image: Chair]
Office Chair
\$299.99
 FURNITURE
Out of Stock

2.4 @Input with Aliases

You can give an input property a different external name using an alias. This is useful when the public API name should differ from the internal property name.

Child Component:

```
export class MessageComponent {
  // External name: 'text', Internal name: 'messageContent'
  @Input('text') messageContent: string = '';

  // External name: 'priority', Internal name: 'level'
  @Input('priority') level: 'low' | 'medium' | 'high' = 'low';
}
```

Parent Template:

```
<!-- Use the alias names in template -->
<app-message
  [text]='Important notification'
  [priority]='high'>
</app-message>
```

Naming Tip: Use aliases sparingly. Only use them when the internal property name would be confusing or when maintaining backwards compatibility with an existing API.

2.5 @Input with Getters and Setters

You can use getters and setters with @Input to perform logic when the input value changes.

Child Component:

```
export class HighlightComponent {
  private _color: string = '';

  @Input()
  set color(value: string) {
    console.log('Color changed to:', value);

    // Validate and sanitize input
    this._color = value || 'yellow';

    // Perform side effects
    this.updateStyles();
  }

  get color(): string {
    return this._color;
  }

  private updateStyles() {
    // Update component styles based on color
    console.log('Updating styles with color:', this._color);
  }
}
```


Usage:

```
<app-highlight [color]='blue'></app-highlight>
<!-- When color is set, the setter runs automatically -->
```

Console Output:

```
Color changed to: blue
Updating styles with color: blue
```

2.6 Required and Optional @Input Properties

In Angular 16+, you can mark inputs as required to enforce that parent components must provide values.

Required Inputs (Angular 16+):

```
export class UserProfileComponent {
  // Required input - parent MUST provide this
  @Input({ required: true }) userId!: number;
  @Input({ required: true }) userName!: string;

  // Optional input - parent can omit this
  @Input() avatarUrl?: string;
  @Input() bio: string = 'No bio available'; // Default value
}
```

Parent Template:

```
<!-- Valid: All required inputs provided -->
<app-user-profile
  [userId]="123"
  [userName]='John Doe'
  [avatarUrl]='assets/avatar.jpg'>
</app-user-profile>

<!-- Valid: Required inputs provided, optional omitted -->
<app-user-profile
  [userId]="456"
  [userName]='Jane Smith'>
</app-user-profile>
```

```
<!-- ERROR: Missing required input 'userId' -->
<!-- <app-user-profile [userName]="Bob"></app-user-profile> -->
```

2.7 Input Transforms (Angular 16+)

Angular 16+ allows you to transform input values automatically using the transform option.

Built-in Transform Functions:

```
import { Component, Input, booleanAttribute, numberAttribute } from '@angular/core';

export class ConfigComponent {
  // Transform string to boolean
  @Input({ transform: booleanAttribute }) enabled: boolean = false;

  // Transform string to number
  @Input({ transform: numberAttribute }) count: number = 0;
}
```

Parent Template:

```
<!-- These string values are automatically converted -->
<app-config enabled="true" count="42"></app-config>
<!-- enabled becomes boolean true, count becomes number 42 -->
```

Custom Transform Function:

```
export class PriceComponent {
  @Input({
    transform: (value: string | number) => {
      return typeof value === 'string' ? parseFloat(value) : value;
    }
  })
  price: number = 0;
}
```

3. Child to Parent Communication with @Output

3.1 Understanding @Output and EventEmitter

The `@Output` decorator marks a class property as an output property. This property must be an `EventEmitter`, which allows the child component to emit custom events that the parent can listen to.

Key Concepts:

- **Event Emission:** Child emits events using `EventEmitter.emit()`
- **Event Binding:** Parent listens using `(eventName)="handler($event)"`
- **Data Transfer:** Child can send data with the event
- **Decoupling:** Child doesn't need to know about parent implementation

3.2 Basic @Output Example

Child Component (`counter.component.ts`):

```
import { Component, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-counter',
  template: `
    <div class="counter">
      <h3>Counter: {{ count }}</h3>
      <button (click)="increment()">Increment</button>
      <button (click)="decrement()">Decrement</button>
      <button (click)="reset()">Reset</button>
    </div>
  `
})
export class CounterComponent {
  count: number = 0;

  // Output event emitter
  @Output() countChanged = new EventEmitter<number>();

  increment() {
    this.count++;
    // Emit the new count value to parent
    this.countChanged.emit(this.count);
  }
}
```

```

}

decrement() {
  this.count--;
  this.countChanged.emit(this.count);
}

reset() {
  this.count = 0;
  this.countChanged.emit(this.count);
}
}

```

Parent Component:

```

export class AppComponent {
  parentCount: number = 0;

  // Handler method that receives emitted value
  onCountChanged(newCount: number) {
    this.parentCount = newCount;
    console.log('Count changed to:', newCount);
  }
}

```

Parent Template:

```

<h2>Parent Component</h2>
<p>Parent knows the count is: {{ parentCount }}</p>

<!-- Listen to child's countChanged event -->
<app-counter (countChanged)="onCountChanged($event)"></app-counter>

```

Parent Component

Parent knows the count is: 5

Counter: 5

[Increment] [Decrement] [Reset]

Console Output:

```
Count changed to: 1
Count changed to: 2
Count changed to: 3
Count changed to: 4
Count changed to: 5
```

3.3 Emitting Complex Data Objects

EventEmitter can emit any type of data, including complex objects.

Interface:

```
interface FormData {
  name: string;
  email: string;
  message: string;
  timestamp: Date;
}
```

Child Component (contact-form.component.ts):

```
import { Component, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-contact-form',
  template: `
    <form (ngSubmit)="submitForm()">
      <input [(ngModel)]="name" name="name" placeholder="Name">
      <input [(ngModel)]="email" name="email" placeholder="Email">
      <textarea [(ngModel)]="message" name="message" placeholder="Message"></textarea>
      <button type="submit">Submit</button>
    </form>
  `
})
export class ContactFormComponent {
  name: string = '';
  email: string = '';
  message: string = '';

  // Emit complex object
```

```
@Output() formSubmitted = new EventEmitter<FormData>();

submitForm() {
  const formData: FormData = {
    name: this.name,
    email: this.email,
    message: this.message,
    timestamp: new Date()
  };

  // Emit the entire object
  this.formSubmitted.emit(formData);

  // Clear form
  this.name = '';
  this.email = '';
  this.message = '';
}
}
```

Parent Component:

```
export class ContactPageComponent {
  submittedForms: FormData[] = [];

  onFormSubmitted(formData: FormData) {
    console.log('Form received:', formData);
    this.submittedForms.push(formData);
  }
}
```

Parent Template:

```
<h2>Contact Us</h2>

<app-contact-form (formSubmitted)="onFormSubmitted($event)"></app-contact-form>

<h3>Received Submissions:</h3>
<div *ngFor="let form of submittedForms">
  <p>Name: {{ form.name | titlecase }}</p>
  <p>Email: {{ form.email }}</p>
  <p>Message: {{ form.message | slice:0:50 }}...</p>
</div>
```

```
<p>Submitted: {{ form.timestamp | date:'short' }}</p>
<hr>
</div>
```

Contact Us

```
[Name Input]
[Email Input]
[Message Textarea]
[Submit Button]
```

Received Submissions:

```
Name: John Doe
Email: john@example.com
Message: I would like to know more about your product...
Submitted: 1/29/26, 2:30 PM
```

3.4 Multiple Output Events

A component can have multiple @Output properties for different types of events.

Child Component (todo-item.component.ts):

```
import { Component, Input, Output, EventEmitter } from '@angular/core';

interface Todo {
  id: number;
  text: string;
  completed: boolean;
}

@Component({
  selector: 'app-todo-item',
  template: `
    <div class="todo-item">
      <input type="checkbox"
        [checked]="todo.completed"
        (change)="onToggle()">
```

```

    <span [class.completed]="todo.completed">{{ todo.text }}</span>
    <button (click)="onEdit()">Edit</button>
    <button (click)="onDelete()">Delete</button>
  </div>
  `,
  styles: [`
    .completed { text-decoration: line-through; color: #999; }
  `]
})
export class TodoItemComponent {
  @Input() todo!: Todo;

  // Multiple output events
  @Output() toggled = new EventEmitter<number>();
  @Output() edited = new EventEmitter<number>();
  @Output() deleted = new EventEmitter<number>();

  onToggle() {
    this.toggled.emit(this.todo.id);
  }

  onEdit() {
    this.edited.emit(this.todo.id);
  }

  onDelete() {
    this.deleted.emit(this.todo.id);
  }
}

```

Parent Component:

```

export class TodoListComponent {
  todos: Todo[] = [
    { id: 1, text: 'Learn Angular', completed: false },
    { id: 2, text: 'Build a project', completed: false },
    { id: 3, text: 'Deploy to production', completed: false }
  ];

  toggleTodo(id: number) {
    const todo = this.todos.find(t => t.id === id);
    if (todo) {
      todo.completed = !todo.completed;
    }
  }
}

```



```

editTodo(id: number) {
  console.log('Edit todo:', id);
  // Open edit dialog
}

deleteTodo(id: number) {
  this.todos = this.todos.filter(t => t.id !== id);
}
}

```

Parent Template:

```

<h2>Todo List</h2>

<app-todo-item
  *ngFor="let todo of todos"
  [todo]="todo"
  (toggled)="toggleTodo($event)"
  (edited)="editTodo($event)"
  (deleted)="deleteTodo($event)">
</app-todo-item>

```

Todo List

- ☐ Learn Angular [Edit] [Delete]
- ☒ Build a project [Edit] [Delete]
- ☐ Deploy to production [Edit] [Delete]

3.5 Emitting Events Without Data

Sometimes you only need to notify the parent that something happened, without sending data.

Child Component:

```
export class ButtonComponent {  
  @Output() clicked = new EventEmitter<void>(); // No data  
  
  handleClick() {  
    this.clicked.emit(); // Emit without data  
  }  
}
```

Parent Template:

```
<app-button (clicked)="handleButtonClick()"></app-button>
```

4. Combining @Input and @Output

4.1 Two-Way Communication Pattern

Components often need both input and output for interactive features.

Child Component (rating.component.ts):

```
import { Component, Input, Output, EventEmitter } from '@angular/core';  
  
@Component({  
  selector: 'app-rating',  
  template: `  
    <div class="rating">  
      <span *ngFor="let star of stars; let i = index"  
        class="star"  
        [class.filled]="i < rating"  
        (click)="rate(i + 1)">  
        {{ i < rating ? '★' : '☆' }}  
      </span>  
      <span class="rating-text">{{ rating }}/{{ maxRating }}</span>  
    </div>  
  `,  
  styles: [`  
    .star {  
      font-size: 24px;  
    }  
  `]  
})
```

```

    cursor: pointer;
    color: #ccc;
  }
  .star.filled {
    color: #ffd700;
  }
  .star:hover {
    color: #ffed4e;
  }
}
`]
})
export class RatingComponent {
  // Input: Receive current rating from parent
  @Input() rating: number = 0;
  @Input() maxRating: number = 5;

  // Output: Emit rating changes to parent
  @Output() ratingChanged = new EventEmitter<number>();

  stars: number[] = [];

  ngOnInit() {
    this.stars = Array(this.maxRating).fill(0);
  }

  rate(value: number) {
    this.rating = value;
    this.ratingChanged.emit(this.rating);
  }
}

```

Parent Component:

```

export class ProductReviewComponent {
  currentRating: number = 0;

  onRatingChanged(newRating: number) {
    this.currentRating = newRating;
    console.log('User rated:', newRating);
  }
}

```

Parent Template:

```
<h2>Rate This Product</h2>
<p>Your Rating: {{ currentRating }}/5</p>

<!-- Both input and output binding -->
<app-rating
  [rating]="currentRating"
  [maxRating]="5"
  (ratingChanged)="onRatingChanged($event)">
</app-rating>
```

Rate This Product

Your Rating: 4/5



Console Output:

User rated: 4

4.2 Two-Way Binding Pattern with Naming Convention

Angular supports two-way binding shortcut syntax when you follow a specific naming pattern:

```
[(propertyName)] .
```

Pattern Rules:

- Input property: `@Input() propertyName`
- Output property: `@Output() propertyNameChange` (must end with "Change")
- Shortcut syntax: `[(propertyName)]="value"`

Child Component (custom-input.component.ts):

```
import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-custom-input',
  template: `
    <div class="custom-input">
```

```

    <label>{{ label }}</label>
    <input
      [value]="value"
      (input)="onInputChange($event)"
      [placeholder]="placeholder">
  </div>
  ,
})
export class CustomInputComponent {
  @Input() label: string = '';
  @Input() placeholder: string = '';

  // Two-way binding pattern
  @Input() value: string = '';
  @Output() valueChange = new EventEmitter<string>(); // Must be 'valueChange'

  onInputChange(event: Event) {
    const newValue = (event.target as HTMLInputElement).value;
    this.value = newValue;
    this.valueChange.emit(newValue);
  }
}

```

Parent Component:

```

export class FormPageComponent {
  userName: string = '';
  userEmail: string = '';
}

```

Parent Template - Long Form:

```

<!-- Traditional way: separate input and output -->
<app-custom-input
  label="Name"
  [value]="userName"
  (valueChange)="userName = $event">
</app-custom-input>

```

Parent Template - Shortcut (Two-Way Binding):

```
<!-- Two-way binding shortcut -->
<app-custom-input
  label="Name"
  placeholder="Enter your name"
  [(value)]= "userName">
</app-custom-input>

<app-custom-input
  label="Email"
  placeholder="Enter your email"
  [(value)]= "userEmail">
</app-custom-input>

<h3>Preview:</h3>
<p>Name: {{ userName }}</p>
<p>Email: {{ userEmail }}</p>
```

Form:**Name****Email****Preview:**

Name: John Doe

Email: john@example.com

Two-Way Binding Syntax: The `[(property)]` syntax is syntactic sugar for `[property]="value" (propertyChange)="value=$event"`. Both work identically.

4.3 Complete Example: Shopping Cart Item

Interface:

```
interface CartItem {
  id: number;
  name: string;
  price: number;
  quantity: number;
  imageUrl: string;
}
```

Child Component (cart-item.component.ts):

```
import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-cart-item',
  template: `
    <div class="cart-item">
      <img [src]="item.imageUrl" [alt]="item.name">

      <div class="item-details">
        <h4>{{ item.name }}</h4>
        <p>Price: {{ item.price | currency:'USD' }}</p>

        <div class="quantity-control">
          <button (click)="decreaseQuantity()">-</button>
          <span>{{ item.quantity }}</span>
          <button (click)="increaseQuantity()">+</button>
        </div>

        <p class="subtotal">
          Subtotal: {{ item.price * item.quantity | currency:'USD' }}
        </p>
      </div>

      <button class="remove-btn" (click)="remove()">x</button>
    </div>
  `,
  styles: [`
    .cart-item {
      display: flex;
      gap: 16px;
      padding: 16px;
      border: 1px solid #e0e0e0;
      margin: 10px 0;
      border-radius: 8px;
    }
  `]
```

```

    }
    .quantity-control { display: flex; gap: 8px; align-items: center; }
    .quantity-control button { width: 30px; height: 30px; }
    .subtotal { font-weight: bold; color: #2c3e50; }
    .remove-btn {
      background: #e74c3c;
      color: white;
      border: none;
      width: 30px;
      height: 30px;
      border-radius: 50%;
      font-size: 20px;
      cursor: pointer;
    }
  `]
})
export class CartItemComponent {
  // Input: Receive item data
  @Input() item!: CartItem;

  // Outputs: Emit different events
  @Output() quantityChanged = new EventEmitter<{id: number, quantity: number}>();
  @Output() itemRemoved = new EventEmitter<number>();

  increaseQuantity() {
    this.item.quantity++;
    this.quantityChanged.emit({
      id: this.item.id,
      quantity: this.item.quantity
    });
  }

  decreaseQuantity() {
    if (this.item.quantity > 1) {
      this.item.quantity--;
      this.quantityChanged.emit({
        id: this.item.id,
        quantity: this.item.quantity
      });
    }
  }

  remove() {
    this.itemRemoved.emit(this.item.id);
  }
}

```


Parent Component:

```
export class ShoppingCartComponent {
  cartItems: CartItem[] = [
    { id: 1, name: 'Laptop', price: 999.99, quantity: 1, imageUrl: 'laptop.jpg' },
    { id: 2, name: 'Mouse', price: 29.99, quantity: 2, imageUrl: 'mouse.jpg' }
  ];

  onQuantityChanged(data: {id: number, quantity: number}) {
    const item = this.cartItems.find(i => i.id === data.id);
    if (item) {
      item.quantity = data.quantity;
    }
    console.log(`Item ${data.id} quantity changed to ${data.quantity}`);
  }

  onItemRemoved(id: number) {
    this.cartItems = this.cartItems.filter(i => i.id !== id);
    console.log(`Item ${id} removed from cart`);
  }

  getTotal(): number {
    return this.cartItems.reduce((sum, item) =>
      sum + (item.price * item.quantity), 0
    );
  }
}
```

Parent Template:

```
<h2>Shopping Cart</h2>

<div class="cart-items">
  <app-cart-item
    *ngFor="let item of cartItems"
    [item]="item"
    (quantityChanged)="onQuantityChanged($event)"
    (itemRemoved)="onItemRemoved($event)">
  </app-cart-item>
</div>
```

```
<div class="cart-total">
  <h3>Total: {{ getTotal() | currency:'USD' }}</h3>
</div>
```

Shopping Cart

[Image] **Laptop** Price: \$999.99 [-] 1 [+] Subtotal: \$999.99



[Image] **Mouse** Price: \$29.99 [-] 2 [+] Subtotal: \$59.98



Total: \$1,059.97

Console Output:

Item 2 quantity changed to 3
 Item 1 removed from cart

5. ViewChild Decorator - Direct Component Access

5.1 Understanding ViewChild

The `@ViewChild` decorator allows a parent component to access a child component instance or DOM element directly. This provides programmatic access to child component methods and properties.

Key Concepts:

- **Direct Access:** Call child component methods from parent
- **Query Timing:** ViewChild is available after `ngAfterViewInit` lifecycle hook
- **Template Reference:** Use template reference variables (`#name`) or component type
- **Read Option:** Specify what to read (component, element, directive)

5.2 Basic ViewChild with Component Type

Child Component (alert.component.ts):

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-alert',
  template: `
    <div *ngIf="isVisible" class="alert" [class]="alertType">
      <p>{{ message }}</p>
      <button (click)="close()">x</button>
    </div>
  `,
  styles: [`
    .alert {
      padding: 12px 16px;
      border-radius: 4px;
      margin: 10px 0;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }
    .success { background: #d4edda; border: 1px solid #c3e6cb; color: #155724; }
    .error { background: #f8d7da; border: 1px solid #f5c6cb; color: #721c24; }
    .warning { background: #fff3cd; border: 1px solid #ffeeba; color: #856404; }
  `]
})
export class AlertComponent {
  message: string = '';
  alertType: 'success' | 'error' | 'warning' = 'success';
  isVisible: boolean = false;

  // Public methods that parent can call
  show(message: string, type: 'success' | 'error' | 'warning' = 'success') {
    this.message = message;
    this.alertType = type;
    this.isVisible = true;
  }

  close() {
    this.isVisible = false;
  }
}
```

Parent Component (app.component.ts):

```
import { Component, ViewChild, AfterViewInit } from '@angular/core';
import { AlertComponent } from './alert/alert.component';

@Component({
  selector: 'app-root',
  template: `
    <h2>ViewChild Example</h2>

    <!-- Child component -->
    <app-alert></app-alert>

    <div class="actions">
      <button (click)="showSuccess()">Show Success</button>
      <button (click)="showError()">Show Error</button>
      <button (click)="showWarning()">Show Warning</button>
    </div>
  `
})
export class AppComponent implements AfterViewInit {
  // Get reference to child component
  @ViewChild(AlertComponent) alertComponent!: AlertComponent;

  ngAfterViewInit() {
    // ViewChild is now available
    console.log('Alert component loaded:', this.alertComponent);
  }

  showSuccess() {
    // Call child component method directly
    this.alertComponent.show('Operation completed successfully!', 'success');
  }

  showError() {
    this.alertComponent.show('An error occurred!', 'error');
  }

  showWarning() {
    this.alertComponent.show('Warning: Check your input!', 'warning');
  }
}
```

ViewChild Example

Operation completed successfully!



[Show Success] [Show Error] [Show Warning]

Console Output:

Alert component loaded: AlertComponent {message: '', alertType: 'success', ...}

5.3 ViewChild with Template Reference Variables

You can use template reference variables to identify which child component or element to access.

Parent Template:

```
<h2>Multiple Child Components</h2>

<!-- Using template reference variables -->
<app-alert #successAlert></app-alert>
<app-alert #errorAlert></app-alert>

<button (click)="showInSuccess()">Show in Success Alert</button>
<button (click)="showInError()">Show in Error Alert</button>
```

Parent Component:

```
import { Component, ViewChild } from '@angular/core';

export class MultipleAlertsComponent {
  // Reference by template variable name
  @ViewChild('successAlert') successAlertComponent!: AlertComponent;
  @ViewChild('errorAlert') errorAlertComponent!: AlertComponent;

  showInSuccess() {
    this.successAlertComponent.show('Success message', 'success');
  }

  showInError() {
```

```

    this.errorAlertComponent.show('Error message', 'error');
  }
}

```

Multiple Child Components

Success message

Error message

[Show in Success Alert] [Show in Error Alert]

5.4 ViewChild with DOM Elements

ViewChild can also reference native DOM elements using template reference variables.

Component TypeScript:

```

import { Component, ViewChild, ElementRef, AfterViewInit } from '@angular/core';

@Component({
  selector: 'app-input-focus',
  template: `
    <h3>Input Focus Example</h3>

    <!-- Template reference variable -->
    <input #nameInput type="text" placeholder="Enter name">
    <input #emailInput type="email" placeholder="Enter email">

    <div class="actions">
      <button (click)="focusName()">Focus Name</button>
      <button (click)="focusEmail()">Focus Email</button>
      <button (click)="clearInputs()">Clear All</button>
    </div>
  `
})
export class InputFocusComponent implements AfterViewInit {
  // Get references to native elements
  @ViewChild('nameInput') nameInputRef!: ElementRef<HTMLInputElement>;
  @ViewChild('emailInput') emailInputRef!: ElementRef<HTMLInputElement>;
}

```

```
ngAfterViewInit() {  
  // Access native element  
  console.log('Name input element:', this.nameInputRef.nativeElement);  
  
  // Auto-focus on first input  
  this.nameInputRef.nativeElement.focus();  
}  
  
focusName() {  
  this.nameInputRef.nativeElement.focus();  
}  
  
focusEmail() {  
  this.emailInputRef.nativeElement.focus();  
}  
  
clearInputs() {  
  this.nameInputRef.nativeElement.value = '';  
  this.emailInputRef.nativeElement.value = '';  
  this.nameInputRef.nativeElement.focus();  
}  
}
```

Input Focus Example

(focused)

[Focus Name] [Focus Email] [Clear All]

Console Output:

Name input element: <input type="text" placeholder="Enter name">

Important: ViewChild queries are set before the `ngAfterViewInit` lifecycle hook. Don't try to access ViewChild in `ngOnInit` or constructor - it will be `undefined`.

5.5 ViewChild with Read Option

When querying elements, you can specify what to read using the `read` option.

Component:

```
import { Component, ViewChild, ElementRef, TemplateRef } from '@angular/core';

@Component({
  selector: 'app-view-child-read',
  template: `
    <div #myDiv>
      <p>Content inside div</p>
    </div>

    <ng-template #myTemplate>
      <p>Template content</p>
    </ng-template>
  `
})
export class ViewChildReadComponent {
  // Read as ElementRef (default for HTML elements)
  @ViewChild('myDiv') divElement!: ElementRef;

  // Read as TemplateRef
  @ViewChild('myTemplate', { read: TemplateRef })
  template!: TemplateRef<any>;

  ngAfterViewInit() {
    console.log('Div element:', this.divElement.nativeElement);
    console.log('Template ref:', this.template);
  }
}
```

5.6 Static ViewChild

By default, ViewChild queries are resolved after view initialization. Use `static: true` to resolve them earlier (in `ngOnInit`) if the queried element is not inside `*ngIf` or `*ngFor`.

Component:


```
export class StaticQueryComponent {
  // Static query: resolved in ngOnInit
  @ViewChild('staticElement', { static: true })
  staticElement!: ElementRef;

  // Dynamic query: resolved in ngAfterViewInit (default)
  @ViewChild('dynamicElement', { static: false })
  dynamicElement!: ElementRef;

  showDynamic = true;

  ngOnInit() {
    // staticElement is available here
    console.log('Static element in ngOnInit:', this.staticElement);

    // dynamicElement is NOT available here yet
    console.log('Dynamic element in ngOnInit:', this.dynamicElement); // undefined
  }

  ngAfterViewInit() {
    // Both are available here
    console.log('Static element in ngAfterViewInit:', this.staticElement);
    console.log('Dynamic element in ngAfterViewInit:', this.dynamicElement);
  }
}
```

Template:

```
<!-- Static element: Always in DOM -->
<div #staticElement>I'm always here</div>

<!-- Dynamic element: Conditionally in DOM -->
<div *ngIf="showDynamic" #dynamicElement>I might not be here</div>
```

Best Practice: Use `static: true` only when necessary (when you need the element in `ngOnInit`). The default `static: false` is safer and works in all situations.

6. ViewChildren - Accessing Multiple Child Components

6.1 Understanding ViewChildren

`@ViewChildren` is used when you need to access multiple child components or elements. It returns a `QueryList`, which is an iterable collection that updates automatically when children are added or removed.

Child Component (tab.component.ts):

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-tab',
  template: `
    <div class="tab-content" *ngIf="active">
      <ng-content></ng-content>
    </div>
  `
})
export class TabComponent {
  @Input() title: string = '';
  @Input() active: boolean = false;

  activate() {
    this.active = true;
  }

  deactivate() {
    this.active = false;
  }
}
```

Parent Component (tabs-container.component.ts):

```
import { Component, ViewChildren, QueryList, AfterViewInit } from '@angular/core';
import { TabComponent } from './tab.component';

@Component({
  selector: 'app-tabs-container',
  template: `
```

```

<div class="tabs">
  <div class="tab-headers">
    <button
      *ngFor="let tab of tabs; let i = index"
      (click)="selectTab(i)"
      [class.active]="i === selectedIndex">
        {{ tab.title }}
      </button>
    </div>

    <div class="tab-content-area">
      <app-tab title="Profile" [active]="true">
        <h3>Profile Information</h3>
        <p>Name: John Doe</p>
        <p>Email: john@example.com</p>
      </app-tab>

      <app-tab title="Settings">
        <h3>Account Settings</h3>
        <p>Notifications: Enabled</p>
        <p>Theme: Dark Mode</p>
      </app-tab>

      <app-tab title="Security">
        <h3>Security Options</h3>
        <p>Two-Factor Auth: Enabled</p>
        <p>Last Login: Today at 2:30 PM</p>
      </app-tab>
    </div>
  </div>
</div>
,
})
export class TabsContainerComponent implements AfterViewInit {
  // Get all TabComponent children
  @ViewChildren(TabComponent) tabs!: QueryList<TabComponent>;

  selectedIndex: number = 0;

  ngAfterViewInit() {
    // Access all child tabs
    console.log('Number of tabs:', this.tabs.length);

    // Activate first tab by default
    if (this.tabs.length > 0) {
      this.selectTab(0);
    }
  }
}

```

```
// Listen to changes in tabs
this.tabs.changes.subscribe(() => {
  console.log('Tabs changed, new count:', this.tabs.length);
});
}

selectTab(index: number) {
  // Deactivate all tabs
  this.tabs.forEach(tab => tab.deactivate());

  // Activate selected tab
  const tabArray = this.tabs.toArray();
  if (tabArray[index]) {
    tabArray[index].activate();
    this.selectedIndex = index;
  }
}
}
```

Tab Interface

Profile	Settings	Security
Profile Information Name: John Doe Email: john@example.com		

Console Output:

Number of tabs: 3

6.2 ViewChildren with Template Reference Variables

Template:

```
<div>
  <input #input1 type="text" value="First">
  <input #input2 type="text" value="Second">
  <input #input3 type="text" value="Third">
</div>
```

```
<button (click)="clearAll()">Clear All</button>
<button (click)="fillAll()">Fill All</button>
</div>
```

Component:

```
import { Component, ViewChildren, QueryList, ElementRef } from '@angular/core';

export class MultipleInputsComponent {
  // Get all input elements
  @ViewChildren('input1, input2, input3')
  inputs!: QueryList<ElementRef<HTMLInputElement>>;

  clearAll() {
    this.inputs.forEach(input => {
      input.nativeElement.value = '';
    });
  }

  fillAll() {
    const values = ['Alpha', 'Beta', 'Gamma'];
    this.inputs.forEach((input, index) => {
      input.nativeElement.value = values[index] || '';
    });
  }

  ngAfterViewInit() {
    console.log('Number of inputs:', this.inputs.length);
  }
}
```

First
Second
Third

[Clear All] [Fill All]

After clicking "Fill All":

Alpha
Beta
Gamma

Console: Number of inputs: 3

7. Real-World Example: Parent-Child-ViewChild Combined

7.1 Accordion Component System

Accordion Item Component (accordion-item.component.ts):

```
import { Component, Input, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-accordion-item',
  template: `
    <div class="accordion-item">
      <div class="accordion-header" (click)="toggle()">
        <h4>{{ title }}</h4>
        <span class="icon">{{ isExpanded ? '-' : '+' }}</span>
      </div>

      <div class="accordion-content" *ngIf="isExpanded">
        <ng-content></ng-content>
      </div>
    </div>
  `,
  styles: [`
    .accordion-item {
      border: 1px solid #ddd;
      margin-bottom: 8px;
      border-radius: 4px;
    }
    .accordion-header {
      padding: 12px 16px;
      background: #f5f5f5;
      cursor: pointer;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }
    .accordion-header:hover {

```

```

    background: #e9e9e9;
  }
  .accordion-content {
    padding: 16px;
    border-top: 1px solid #ddd;
  }
  .icon {
    font-size: 20px;
    font-weight: bold;
  }
  `]
})
export class AccordionItemComponent {
  // Input: Receive configuration from parent
  @Input() title: string = '';
  @Input() isExpanded: boolean = false;

  // Output: Notify parent when toggled
  @Output() toggled = new EventEmitter<void>();

  // Public method that parent can call via ViewChild
  toggle() {
    this.isExpanded = !this.isExpanded;
    this.toggled.emit();
  }

  expand() {
    this.isExpanded = true;
  }

  collapse() {
    this.isExpanded = false;
  }
}

```

Parent Component (faq.component.ts):

```

import { Component, ViewChildren, QueryList } from '@angular/core';
import { AccordionItemComponent } from './accordion-item.component';

@Component({
  selector: 'app-faq',
  template: `
    <h2>Frequently Asked Questions</h2>

```

```

<div class="controls">
  <button (click)="expandAll()">Expand All</button>
  <button (click)="collapseAll()">Collapse All</button>
</div>

<app-accordion-item
  [title]='What is Angular?'"
  [isExpanded]="true"
  (toggled)="onItemToggled(0)">
  <p>Angular is a TypeScript-based web application framework.</p>
</app-accordion-item>

<app-accordion-item
  [title]='How to install Angular?'"
  (toggled)="onItemToggled(1)">
  <p>Install Angular CLI using: npm install -g @angular/cli</p>
</app-accordion-item>

<app-accordion-item
  [title]='What are components?'"
  (toggled)="onItemToggled(2)">
  <p>Components are the building blocks of Angular applications.</p>
</app-accordion-item>

<app-accordion-item
  [title]='What is dependency injection?'"
  (toggled)="onItemToggled(3)">
  <p>DI is a design pattern for managing dependencies in applications.</p>
</app-accordion-item>
,
})
export class FaqComponent {
  // Get all accordion items
  @ViewChildren(AccordionItemComponent)
  accordionItems!: QueryList<AccordionItemComponent>;

  expandAll() {
    // Use ViewChildren to call method on all items
    this.accordionItems.forEach(item => item.expand());
  }

  collapseAll() {
    this.accordionItems.forEach(item => item.collapse());
  }

  onItemToggled(index: number) {

```



```
    console.log(`Accordion item ${index} toggled`);  
  }  
}
```

Frequently Asked Questions

[Expand All] [Collapse All]

What is Angular?

—

Angular is a TypeScript-based web application framework.

How to install Angular?

+

What are components?

+

Console Output:

```
Accordion item 0 toggled  
Accordion item 1 toggled
```

6.2 QueryList Methods and Properties

The `QueryList` returned by `ViewChildren` has useful methods and properties.

Component Example:

```
export class QueryListExampleComponent {  
  @ViewChildren(TabComponent) tabs!: QueryList<TabComponent>;  
  
  ngAfterViewInit() {  
    // QueryList properties  
    console.log('Length:', this.tabs.length);  
    console.log('First tab:', this.tabs.first);  
    console.log('Last tab:', this.tabs.last);  
  }  
}
```

```
// Convert to array
const tabsArray = this.tabs.toArray();
console.log('As array:', tabsArray);

// Iterate with forEach
this.tabs.forEach((tab, index) => {
  console.log(`Tab ${index}:`, tab.title);
});

// Find specific tab
const foundTab = this.tabs.find(tab => tab.title === 'Settings');

// Filter tabs
const activeTabs = this.tabs.filter(tab => tab.active);

// Listen to changes
this.tabs.changes.subscribe((newTabs: QueryList<TabComponent>) => {
  console.log('Tabs collection changed:', newTabs.length);
});
}
```

8. Communication Comparison Table

Feature	@Input	@Output	ViewChild
Direction	Parent → Child	Child → Parent	Parent → Child
Data Flow	Pass data down	Emit events up	Direct access
Coupling	Loose (declarative)	Loose (event-based)	Tight (imperative)
When Available	Immediately	Immediately	After ngAfterViewInit
Use Case	Pass configuration/ data	Notify about events	Call child methods directly
Syntax	[property]="value"	(event)="handler(\$event)"	@ViewChild(Type) child!
Best For	Data binding	Event handling	Programmatic control

9. Best Practices and Common Patterns

9.1 When to Use Each Method

Use @Input when:

- Passing configuration or data to child components
- You need declarative, template-based data binding
- The data flows one-way from parent to child
- You want loose coupling between components

Use @Output when:

- Child needs to notify parent about events
- You want event-driven communication
- Child shouldn't know about parent implementation
- Multiple parents might handle the same event differently

Use ViewChild when:

- Parent needs direct access to child component methods
- You need programmatic control over child behavior
- Working with third-party components without @Output events
- Accessing native DOM elements for imperative operations

Avoid ViewChild when:

- @Input or @Output would work better (prefer declarative over imperative)
- You can achieve the same result with data binding

- It would create tight coupling that makes components hard to reuse

9.2 Input Property Validation

Validating Input Values:

```
export class ValidatedInputComponent {
  private _percentage: number = 0;

  @Input()
  set percentage(value: number) {
    // Validate input
    if (value < 0 || value > 100) {
      console.warn('Percentage must be between 0 and 100');
      this._percentage = Math.max(0, Math.min(100, value));
    } else {
      this._percentage = value;
    }
  }

  get percentage(): number {
    return this._percentage;
  }
}
```

9.3 Avoiding Memory Leaks with @Output

Good Practice - EventEmitter Cleanup:

```
export class CleanupComponent implements OnDestroy {
  @Output() dataChanged = new EventEmitter<string>();

  private interval: any;

  ngOnInit() {
    // Emit events periodically
    this.interval = setInterval(() => {
      this.dataChanged.emit('Update');
    }, 1000);
  }
}
```

```
ngOnDestroy() {  
  // Clean up interval when component is destroyed  
  if (this.interval) {  
    clearInterval(this.interval);  
  }  
  
  // Complete the EventEmitter (optional but recommended)  
  this.dataChanged.complete();  
}  
}
```

9.4 OnChanges Hook with @Input

Use `ngOnChanges` to react to input property changes.

Component:

```
import { Component, Input, OnChanges, SimpleChanges } from '@angular/core';  
  
export class InputChangeComponent implements OnChanges {  
  @Input() userId!: number;  
  @Input() userName!: string;  
  
  userDetails: any = null;  
  
  ngOnChanges(changes: SimpleChanges) {  
    console.log('Input changes detected:', changes);  
  
    // Check specific property changes  
    if (changes['userId']) {  
      console.log('UserId changed from',  
        changes['userId'].previousValue,  
        'to',  
        changes['userId'].currentValue  
      );  
  
      // Fetch new user details  
      this.loadUserDetails(changes['userId'].currentValue);  
    }  
  
    if (changes['userName']) {  
      console.log('UserName changed:', changes['userName'].currentValue);  
    }  
  }  
}
```

```
}

loadUserDetails(userId: number) {
  // Simulate API call
  console.log('Loading details for user:', userId);
}
}
```

Parent Template:

```
<app-input-change [userId]="selectedUserId" [userName]="selectedUserName">
</app-input-change>

<button (click)="selectedUserId = 123">Change User</button>
```

Console Output:

```
Input changes detected: {userId: SimpleChange, userName: SimpleChange}
UserId changed from undefined to 123
Loading details for user: 123
UserName changed: John Doe
```

10. Practice Exercises

Exercise 1: Create a Reusable Button Component

Build a button component that:

- Accepts @Input for label, type (primary/secondary/danger), and disabled state
- Emits @Output event when clicked
- Parent component uses 3 different button instances with different configurations
- Show how parent handles each button click differently

Exercise 2: Build a Temperature Converter

Create parent-child components where:

- Child component displays temperature in Celsius with input field
- Child emits @Output event when temperature changes
- Parent receives Celsius value and calculates Fahrenheit and Kelvin
- Parent passes calculated values back to display components via @Input

Exercise 3: Implement a Modal Dialog System

Build a modal component that:

- Parent uses ViewChild to open/close modal programmatically
- Modal accepts @Input for title and content
- Modal emits @Output when user clicks confirm or cancel
- Parent shows different modals based on user actions

Exercise 4: Create a Filterable Product List

Build a system with:

- Filter component (child) that accepts @Input for available filters
- Filter component emits @Output when user selects filters
- Product list component (child) that accepts @Input for filtered products
- Parent component coordinates between filter and product list
- Parent uses ViewChild to reset filters programmatically

Exercise 5: Build a Wizard/Stepper Component

Create a multi-step wizard where:

- Each step is a child component with @Input for step data
- Each step emits @Output when user clicks next/previous
- Parent uses ViewChildren to access all steps
- Parent can validate all steps, jump to specific step, or navigate sequentially

- Include progress indicator showing current step

11. Common Mistakes and Pitfalls

Mistake 1: Accessing ViewChild in ngOnInit

```
// ✗ Wrong
ngOnInit() {
  this.myChild.doSomething(); // undefined!
}

// ✓ Correct
ngAfterViewInit() {
  this.myChild.doSomething(); // Available now
}
```

Mistake 2: Forgetting to Emit Events

```
// ✗ Wrong
@Output() clicked = new EventEmitter<void>();

handleClick() {
  // Forgot to emit!
  console.log('Clicked');
}

// ✓ Correct
handleClick() {
  this.clicked.emit(); // Emit the event
}
```

Mistake 3: Mutating @Input Objects Directly


```
// ✗ Wrong (modifying parent's object)
@Input() user!: User;

updateUser() {
  this.user.name = 'New Name'; // Mutates parent's object!
}

// ☑ Correct (emit event to parent)
@Input() user!: User;
@Output() userUpdated = new EventEmitter<User>();

updateUser() {
  const updatedUser = { ...this.user, name: 'New Name' };
  this.userUpdated.emit(updatedUser);
}
```

Mistake 4: Not Typing EventEmitter

```
// ✗ Wrong (no type)
@Output() changed = new EventEmitter();

// ☑ Correct (with type)
@Output() changed = new EventEmitter<string>();
```

12. Summary

In this module, you learned:

- **@Input Decorator:** Passing data from parent to child components with property binding
- **@Output Decorator:** Emitting events from child to parent using EventEmitter
- **ViewChild:** Direct access to child components and DOM elements
- **ViewChildren:** Accessing multiple child components with QueryList
- **Combined Patterns:** Using @Input and @Output together for interactive components

- **Two-Way Binding:** Implementing [(value)] pattern with Input and Output
- **Best Practices:** When to use each method, validation, and error prevention

These communication patterns form the foundation for building interactive, reusable Angular components. Mastering @Input, @Output, and ViewChild is essential for creating component-based architectures.

Next Module Preview: In Module 8.2, we'll explore Event Handling patterns, including custom event binding, EventEmitter vs Subject/Observable, and advanced event management techniques.

Module 8.1 - Component Communication Overview | Angular Framework Training

